به نام خدا

دانشگاه تهران

آزمایشگاه سیستم‌عامل - پاییز ۱۴۰۵

گزارش پروژه اول: آشنایی با هسته‌ی سیستم‌عامل xv۶

اعضای گروه: مجید احسانی، دنیز نجفی، کیان پیامی‌مهر

شماره دانشجویی‌ها: 810103376- 810101589 - 810103347

نام فایل تحویلی: OS-lab1-810103347-810101589-810103376.zip

بخش اول: آشنایی با سیستم‌عامل و xv۶

سوال 1: سه وظیفه اصلی سیستم‌عامل را نام ببرید.

- **مدیریت منابع:** پردازنده، حافظه و دستگاه‌های ورودی/خروجی بین برنامه‌ها تقسیم می‌شود (زمان‌بندی پردازه‌ها، تخصیص و آزادسازی حافظه و ...).
- **ایجاد انتزاع و رابط برای برنامه‌ها:** مثلاً به جای کار با سکتورهای دیسک، مفهوم فایل و پوشه را می‌دهد و از طریق فراخوانی‌های سیستمی سرویس ارائه می‌کند.
- **حفاظت و جداسازی:** نمی‌گذارد یک برنامه به حافظه‌ی برنامه‌ی دیگر یا هسته دست بزند و خرابی یک برنامه کل سیستم را از کار نیندازد.

سوال 2: آیا وجود سیستم‌عامل در تمام سیستم‌های کامپیوتری الزامی است؟ چرا؟ مزایای آن نسبت به دسترسی مستقیم برنامه‌ها به سخت‌افزار چیست؟

نه، الزامی نیست. در سیستم‌های خیلی ساده مثل بسیاری از میکروکنترلرها و سیستم‌های نهفته، یک برنامه‌ی تکی مستقیم روی سخت‌افزار اجرا می‌شود (bare-metal) و چون فقط یک کار انجام می‌دهد نیازی به مدیریت منابع بین چند برنامه نیست. سیستم‌عامل زمانی لازم می‌شود که چند برنامه و چند کاربر بخواهند از یک سخت‌افزار مشترک استفاده کنند.

مزایای سیستم‌عامل نسبت به دسترسی مستقیم:

- برنامه‌نویس لازم نیست جزئیات هر سخت‌افزار را بداند و برنامه روی سخت‌افزارهای مختلف قابل اجراست (قابلیت حمل).
- چند برنامه می‌توانند همزمان و منصفانه از پردازنده، حافظه و دیسک استفاده کنند.
- امنیت و ایزوله‌سازی: برنامه‌ها به‌طور مستقیم به سخت‌افزار دسترسی ندارند و باید از هسته درخواست بدهند.
- از تکرار کد جلوگیری می‌شود، چون درایورها و سرویس‌های مشترک یک بار در سیستم‌عامل نوشته شده‌اند.

سوال 3: چه معماری‌هایی برای سازمان‌دهی هسته وجود دارد؟ xv۶ چه معماری‌ای دارد؟

- **یکپارچه (Monolithic)**: تمام سرویس‌های سیستم‌عامل (زمان‌بندی، حافظه، فایل‌سیستم، درایورها) داخل یک برنامه‌ی بزرگ در حالت هسته اجرا می‌شوند و مستقیم توابع هم را صدا می‌زنند. سریع است ولی خطا در یک بخش می‌تواند کل هسته را از کار بیندازد.
- **ریزهسته (Microkernel)**: فقط کارهای حداقلی (ارتباط بین پردازه‌ها، زمان‌بندی پایه و مدیریت حافظه‌ی پایه) در هسته است و بقیه‌ی سرویس‌ها مثل فایل‌سیستم و درایورها به صورت پردازه‌های سطح کاربر اجرا می‌شوند. امن‌تر و ماژولارتر است اما به خاطر پیام‌رسانی زیاد کندتر است.
- **ترکیبی (Hybrid)**: حالتی بین این دو؛ ساختار کلی ریزهسته است ولی برای سرعت، بعضی سرویس‌ها داخل هسته آمده‌اند (مثل ویندوز NT و macOS).
- **لایه‌ای (Layered) و اگزوکرنل (Exokernel)**: در معماری لایه‌ای سیستم‌عامل به چند لایه تقسیم می‌شود که هر لایه فقط از لایه‌ی زیرین استفاده می‌کند؛ در اگزوکرنل هسته فقط منابع را محافظت می‌کند و مدیریت آن‌ها را به برنامه‌ها می‌سپارد.

xv۶ از نظر سازمان‌دهی هسته **یکپارچه (Monolithic)** است؛ همه‌ی بخش‌های آن (پردازه‌ها، حافظه، فایل‌سیستم، pipe، درایور دیسک و کنسول) در یک برنامه‌ی هسته (فایل `kernel`) و در یک فضای آدرس و در بالاترین سطح دسترسی اجرا می‌شوند.

سوال 4: xv۶ تک‌وظیفه‌ای است یا چندوظیفه‌ای؟

xv۶ **چندوظیفه‌ای (Multitasking)** است. هسته می‌تواند چند پردازه را همزمان در حافظه نگه دارد و با کمک وقفه‌ی تایمر، پردازنده را بین آن‌ها تقسیم کند (اشتراک زمانی). مثلاً وقتی شل در حال اجراست، پردازه‌های دیگر مثل `init` یا برنامه‌هایی که از شل اجرا می‌کنیم هم می‌توانند وجود داشته باشند. (البته هر پردازه فقط یک ریسه دارد.)

سوال 5: تفاوت برنامه و پردازه چیست؟

برنامه یک فایل روی دیسک است (کد و داده‌ی ثابت) و خودش کاری نمی‌کند؛ مثلاً فایل `ls`. پردازه یک برنامه‌ی **در حال اجراست** که حافظه‌ی خودش، ثبات‌ها، شناسه‌ی پردازه (pid)، فایل‌های باز و وضعیت اجرا را دارد. از یک برنامه می‌توان چند پردازه‌ی مستقل ساخت (مثلاً دو بار اجرای `ls` دو پردازه‌ی جدا هستند).

سوال 6: ساختار یک پردازه در xv۶ از چه بخش‌هایی تشکیل شده است؟ پردازنده چگونه به پردازه‌ها اختصاص داده می‌شود؟

هر پردازه در xv۶ از دو بخش اصلی تشکیل شده است:

- **حافظه‌ی کاربر:** از آدرس صفر شروع می‌شود و شامل کد (text) و داده‌ی برنامه، بعد یک صفحه‌ی محافظ (guard page)، پشته‌ی کاربر و بالاتر از آن heap است.
- **وضعیت داخل هسته:** ساختار `struct proc` شامل اندازه‌ی حافظه (`sz`)، جدول صفحات (`pgdir`)، پشته‌ی هسته (`kstack`)، حالت پردازه (`state`: UNUSED, EMBRYO, SLEEPING, RUNNABLE, RUNNING, ZOMBIE)، `pid`، اشاره‌گر به پدر (`parent`)، `trapframe` (ثبات‌های ذخیره‌شده هنگام ورود به هسته)، `context` (برای تعویض متن در هسته)، جدول فایل‌های باز (`ofile`)، پوشه‌ی جاری (`cwd`) و نام پردازه.

برای اختصاص پردازنده، xv۶ از زمان‌بندی **نوبت‌گردشی** (**Round-Robin**) استفاده می‌کند. هر پردازنده یک حلقه‌ی `scheduler()` دارد که جدول پردازه‌ها (`ptable`) را می‌گردد و اولین پردازه‌ی RUNNABLE را پیدا می‌کند و با `swtch` به آن می‌رود. وقتی وقفه‌ی تایمر بیاید، پردازه با `yield()` پردازنده را رها می‌کند و دوباره به `scheduler` برمی‌گردیم تا پردازه‌ی بعدی انتخاب شود. پردازه‌ای که منتظر I/O است هم با `sleep` پردازنده را رها می‌کند.

سوال 7: Descriptor File چیست؟ عملکرد Pipe در xv۶ چگونه است و برای چه استفاده می‌شود؟

در سیستم‌های یونیکسی هر چیزی (فایل، کنسول، pipe، دستگاه) از دید برنامه شبیه یک فایل است که با یک عدد صحیح کوچک به نام **file descriptor** به آن اشاره می‌شود. این عدد اندیس یک آرایه به نام `ofile` در `struct proc` است که به `struct file` در هسته اشاره می‌کند. به صورت قرارداد، ۰ ورودی استاندارد، ۱ خروجی استاندارد و ۲ خروجی خطاست. با `read`، `write` و `close` روی همین عدد کار می‌کنیم و برنامه لازم نیست بداند پشتش فایل است یا کنسول.

Pipe یک بافر کوچک در هسته است (در xv۶ اندازه‌اش ۵۱۲ بایت، `PIPESIZE`) که یک سر برای نوشتن و یک سر برای خواندن دارد. فراخوانی `pipe()` دو descriptor برمی‌گرداند. هرچه در سر نوشتن نوشته شود به ترتیب از سر خواندن بیرون می‌آید. اگر بافر پر شود نویسنده و اگر خالی باشد خواننده `sleep` می‌شود تا طرف مقابل کارش را انجام دهد. اگر همه‌ی سرهای نوشتن بسته شوند، خواننده EOF می‌گیرد. کاربرد معمول آن ارتباط بین پردازه‌هاست، مثلاً در شل برای دستور `cat README | wc` خروجی `cat` به ورودی `wc` وصل می‌شود.

سوال 8: فراخوانی‌های fork و exec چه می‌کنند؟ ادغام نکردن آن‌ها چه مزیتی دارد؟

- `fork`: یک پردازه‌ی جدید می‌سازد که کپی پردازه‌ی فعلی است (حافظه، فایل‌های باز و ...). در پدر مقدار برگشتی pid فرزند و در فرزند ۰ است.
- `exec`: حافظه‌ی پردازه‌ی فعلی را با یک برنامه‌ی جدید که از فایل ELF خوانده می‌شود جایگزین می‌کند (pid و فایل‌های باز می‌مانند ولی کد و داده عوض می‌شود). در صورت موفقیت هیچ‌وقت برنمی‌گردد.

مزیت جدا بودنشان این است که بین این دو فراخوانی فرزند فرصت دارد محیط خودش را تغییر دهد. مثلاً شل بعد از `fork` در فرزند descriptorها را عوض می‌کند (با `close` و `open` برای ریدایرکت `<` یا وصل کردن pipe) و بعد `exec` می‌زند. اگر این دو یکی بودند باید برای هر حالت ریدایرکت و pipe پارامترهای زیادی به آن داده می‌شد یا شل خودش را تغییر می‌داد. همچنین `fork` به‌تنهایی هم کاربرد دارد (مثلاً سرور که چند کپی از خودش می‌سازد).

بخش دوم: اجرا و برپایی xv۶ روی QEMU

سوال 9: دستوری که فایل نهایی هسته را می‌سازد پیدا کنید. از چه ابزاری استفاده می‌کند؟

بعد از `make clean` و اجرای `make -n` دستور زیر برای ساخت فایل `kernel` دیده می‌شود:

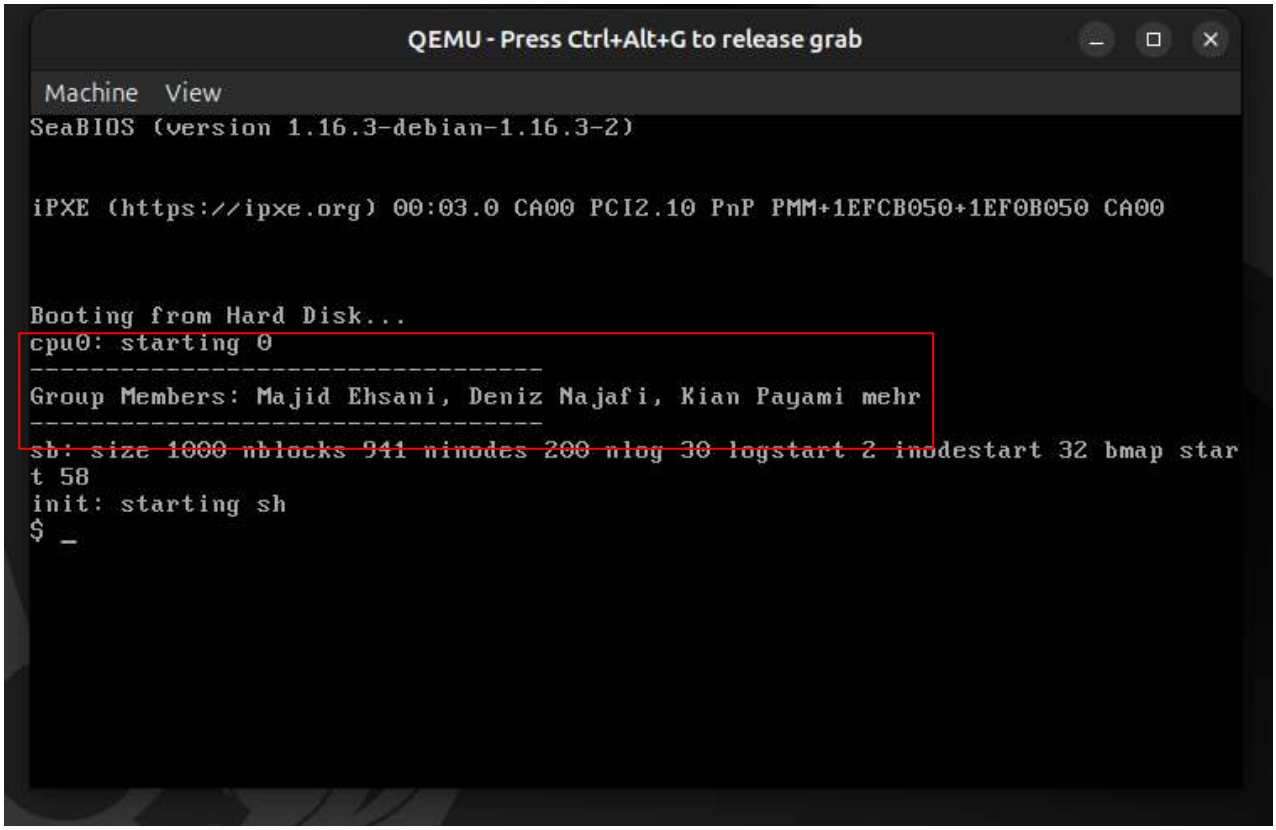
```
ld -m elf_i386 -T kernel.ld -o kernel entry.o bio.o console.o exec.o file.o fs.o \
 ide.o ioapic.o kalloc.o kbd.o lapic.o log.o main.o mp.o picirq.o pipe.o proc.o \
 sleeplock.o spinlock.o string.o swtch.o syscall.o sysfile.o sysproc.o trapasm.o \
 trap.o uart.o vectors.o vm.o -b binary initcode entryother
```

این دستور از **لینکر** `(GNU ld)` **`ld`** استفاده می‌کند. همه‌ی فایل‌های شیء `.o` هسته را طبق اسکریپت `kernel.ld` کنار هم می‌گذارد و آدرس‌ها را تعیین می‌کند. گزینه‌ی `-b binary` باعث می‌شود فایل‌های `initcode` و `entryother` هم به صورت خام داخل خود هسته قرار بگیرند. بعد از آن `objdump` هم برای ساخت `kernel.asm` و `kernel.sym` اجرا می‌شود.

بخش سوم: اضافه کردن متن به پیام بوت

برای نمایش نام اعضای گروه، فایل **`main.c`** و داخل تابع **`mpmain`**`()` را تغییر دادیم. بعد از خط `cprintf("cpu%d: starting ...")` و قبل از `idtinit()`، چند خط `cprintf` اضافه کردیم. این کد داخل شرط `if (cpuid() == 0)` است تا فقط پردازنده‌ی اصلی (BSP) پیام را چاپ کند و در حالت چندپردازنده‌ای چند بار تکرار نشود. چون `mpmain` قبل از اجرای پردازه‌ی `init` و شل اجرا می‌شود، این پیام قبل از خط فرمان (`$`) روی کنسول دیده می‌شود.

```
  if (cpuid() == 0) {
    cprintf("---------------------------------\n");
    cprintf("Group Members: Majid Ehsani, Deniz Najafi, Kian Payami mehr\n");
    cprintf("---------------------------------\n");
  }
```



بخش چهارم: قابلیت‌های جدید کنسول و پوسته

قابلیت‌های حرکت مکان‌نما، انتخاب/کپی/جایگذاری در فایل **`console.c`** (تابع `consoleintr`) و قابلیت‌های Alias و History در **`sh.c`** پیاده‌سازی شده‌اند. در ادامه خلاصه‌ی پیاده‌سازی و بعد Caseهای تست آورده شده است.

خلاصه‌ی پیاده‌سازی

- **حرکت مکان‌نما:** در `struct input` متغیر `pos` نگه‌داری می‌شود. کلیدهای ← و → و Ctrl+Q و Ctrl+E با تابع `movecursor` مقدار `pos` را در بازه‌ی `w ≤ pos ≤ e` عوض می‌کنند. Backspace کاراکتر قبل از `pos` را حذف می‌کند و بقیه‌ی بافر را یک خانه به عقب می‌آورد (`removerange`)، و درج کاراکتر هم با `insertchars` وسط بافر انجام می‌شود.
- **انتخاب/کپی/پیست:** با Ctrl+I حالت انتخاب (`selstate`) روشن می‌شود و `selbeg` ذخیره می‌شود. بعد از Ctrl+I دوم `selend` ثبت می‌شود و بازه‌ی بین این دو (با هر ترتیبی، تابع `getsel`) هایلایت می‌شود. کپی در آرایه‌ی `clip` ذخیره می‌شود. تابع `redraw` کل خط را از روی بافر دوباره روی صفحه می‌کشد تا متن نمایش‌داده‌شده همیشه با بافر یکی باشد.
- **Alias:** در `sh.c` آرایه‌ی `aliases` (تا ۱۶ مورد، بیشتر از حداقل ۱۰ تا) و توابع `setalias` و `expandalias`. فقط کلمه‌ی اول دستور یک بار جایگزین می‌شود.
- **History:** آرایه‌ی حلقوی `hist` با اندازه‌ی ۱۰ و شمارنده‌ی `histnext` که همیشه زیاد می‌شود، پس شماره‌ی دستورها با پاک شدن قدیمی‌ها عوض نمی‌شود. تابع `histexpand` دستورهای `!!` و `!n` و `!prefix` را باز می‌کند و ابتدا history و بعد alias اعمال می‌شود.

Caseهای تست

در جدول زیر Caseهایی که برای آزمایش استفاده کردیم و دلیل انتخاب هر کدام آمده است. (← و → یعنی کلیدهای جهت‌دار.)

الف) جابه‌جایی مکان‌نما و حذف

#	ورودی	خروجی مورد انتظار	دلیل انتخاب
۱	تایپ `hello` ← ← تایپ X	`helXlo`	درج کاراکتر وسط رشته و جابه‌جایی بقیه‌ی متن
۲	تایپ `abc` ، Ctrl+Q ، تایپ Z ، Ctrl+E ، تایپ Y	`ZabcY`	درستی پرش به ابتدا و انتهای خط
۳	خط خالی: ← و Backspace	بدون تغییر	شرط مرزی ابتدای رشته
۴	تایپ `abc` و فشردن →	بدون تغییر	شرط مرزی انتهای رشته
۵	تایپ `abcd` ← ← Backspace	`acd` و مکان‌نما بعد از a	Backspace باید قبل از مکان‌نما را حذف کند نه آخر خط را

#	ورودی	خروجی مورد انتظار	دلیل انتخاب
۶	تایپ abcd ← ← Backspace Backspace Backspace	cd (سومین Backspace بی‌اثر)	Backspace در ابتدای خط نباید به بافر یا خط قبلی آسیب بزند

ب) انتخاب، کپی و جایگذاری

#	ورودی	خروجی مورد انتظار	دلیل انتخاب
۷	تایپ hello world ، Ctrl+Q ، Ctrl+I ، Ctrl+I ، Backspace ، ۶× →	world (هر شش کاراکتر «hello» و فاصله حذف می‌شود)	انتخاب و حذف کل بخش انتخابی
۸	همان پایان خط: Ctrl+I ، ۵× ← ، Ctrl+I	world هایلایت می‌شود	انتخاب برعکس (نقطه‌ی دوم قبل از اول)
۹	Ctrl+I و بلافاصله Ctrl+I دوباره	هیچ انتخابی ایجاد نمی‌شود	شروع و پایان یکسان
۱۰	انتخاب Ctrl+V ، Ctrl+C ، world (بدون حرکت)	Ctrl+C ؛ انتخاب باhello world آن را با Ctrl+V باقی می‌ماند و همان متن کپی‌شده جایگزین می‌کند و انتخاب تمام می‌شود	کپی نباید انتخاب را بردارد؛ Ctrl+V باید انتخاب را با clip جایگزین کند
۱۰ب	انتخاب Ctrl+E ، Ctrl+C ، world (حرکت)، Ctrl+V	؛ حرکتhello worldworld مکان‌نما انتخاب را تمام می‌کند و محتوای کپی‌شده در محل مکان‌نما درج می‌شود	رفتار Ctrl+V بعد از پایان انتخاب (حالت بدون انتخاب)
۱۱	بدون انتخاب: Ctrl+V	محتوای کپی‌شده در محل مکان‌نما درج می‌شود	حالت بدون انتخاب
۱۲	انتخاب یک بخش و تایپ Q	بخش انتخاب‌شده با Q عوض می‌شود و انتخاب تمام می‌شود	جایگزینی با کاراکتر جدید
۱۳	انتخاب، سپس Enter	دستور با متن کامل و صحیح اجرا می‌شود	سازگاری بافر ورودی با متن روی صفحه

ج) Alias و History و ترکیب آن‌ها

#	ورودی	خروجی مورد انتظار	دلیل انتخاب
۱۴	سپس alias search=grep search the README	همان خروجی grep the README	جایگزینی کلمه‌ی اول و حفظ آرگومان‌ها
۱۵	alias x=echo a سپس alias x=echo b و x	b	تعریف مجدد باید مقدار قبلی را عوض کند
۱۶	تعریف ۱۰ alias مختلف و استفاده از اولی و آخری	همه کار کنند	حداقل ۱۰ alias همزمان

#	ورودی	خروجی مورد انتظار	دلیل انتخاب
۱۷	`wc ،cat README ،echo hello history ،README`	`2. cat README ،1. echo hello` `history` و بعد خود `3. wc README ،` به‌عنوان `4.`	نمایش شماره‌دار و ترتیبی
۱۸	`!ec ،!2 ،!!`	دستور گسترش‌یافته چاپ و اجرا می‌شود	سه نوع Expansion
۱۹	اجرای بیش از ۱۰ دستور و سپس `history`	فقط ۱۰ دستور آخر و شماره‌ها ادامه‌دار (مثلاً از ۴ تا ۱۳)	شماره‌ها پس از حذف قدیمی‌ها تغییر نکنند
۲۰	`!zzz`	پیام خطا و عدم اجرای دستور	دستور نامعتبر در history
۲۱	`!! ،e hi ،alias e=echo`	`e hi` چاپ شده و بعد خروجی `hi`	ابتدا History و بعد Alias
۲۲	تایپ `ech` ، ← ، تایپ `o` ، رفتن به انتها، `hi` و Enter؛ بعد کپی `hi` و پیست در `echo` و Enter	`echo hi` و بعد `echo hi` دوباره اجرا می‌شود	تست ترکیبی حرکت مکان‌نما، کپی/پیست و اجرا

بخش پنجم: برنامه‌ی سطح کاربر rle

برنامه در فایل **`rle.c`** نوشته شد و به `Makefile` (هم در `UPROGS` و هم در `EXTRA`) اضافه شد (`_rle`). برنامه با `open`، `read`، `write` و `close` کار می‌کند. در حالت `encode` هر دنباله را به شکل `;`کاراکتر`,`تعداد در `encoded.txt` و در حالت `decode` متن اصلی را در `decoded.txt` می‌نویسد. برای فایل خالی، حالت نامعتبر، تعداد آرگومان اشتباه و فایل غیرقابل باز شدن پیام خطا می‌دهد (مثلاً `usage: rle encode|decode file`).

#	ورودی	خروجی مورد انتظار	دلیل انتخاب
۱	`aaabbccccdd` و encode	`a,3;b,2;c,4;d,2;`	نمونه‌ی صورت پروژه
۲	decode همان فایل	`aaabbccccdd`	encode و decode باید معکوس هم باشند
۳	فایل خالی	فایل خروجی خالی و بدون خطا	شرط مرزی
۴	یک کاراکتر تکی مثل `a`	`a,1;`	تعداد ۱
۵	`rle encode` و `rle foo input.txt` و فایل ناموجود	پیام خطای مناسب و خروج	اعتبارسنجی ورودی

سوال 10: در Makefile متغیرهای UPROGS و ULIB چه کاربردی دارند؟

- **`ULIB`** فایل‌های شیء کتابخانه‌ی سطح کاربر را نگه می‌دارد: `ulib.o usys.o printf.o umalloc.o`. شامل توابع کمکی مثل `strcpy`، `printf`، `malloc` و کد فراخوانی‌های سیستمی (`usys.S`) است. قاعده‌ی `_%: %.o $(ULIB)` باعث می‌شود هر برنامه‌ی کاربر هنگام لینک با این کتابخانه ترکیب شود.

- **`UPROGS`** لیست برنامه‌های سطح کاربر است (`_cat`، `_ls`، `_sh`، `_rle` و ...). `mkfs` همه‌ی این‌ها را داخل تصویر دیسک `fs.img` می‌گذارد تا داخل xv۶ قابل اجرا باشند. برای همین برای اضافه کردن `rle` باید `_rle` را در این لیست می‌نوشتیم.

سوال 11: چرا برای `cd` فایل اجرایی جدا وجود ندارد؟ در کجا پیاده شده؟

`cd` به صورت داخلی (built-in) در خود شل و در تابع `main` فایل **`sh.c`** پیاده شده است: شل خط ورودی را که با `cd` شروع شود می‌گیرد و مستقیم فراخوانی سیستمی `chdir` را صدا می‌زند. خود `chdir` در هسته در تابع `sys_chdir` فایل `sysfile.c` است و `cwd` پردازه‌ی فعلی را به inode پوشه‌ی جدید تغییر می‌دهد.

علت اینکه مثل `ls` در پردازه‌ی فرزند اجرا نمی‌شود این است که پوشه‌ی جاری یک ویژگی **هر پردازه** است. اگر `cd` در فرزند اجرا می‌شد، فقط `cwd` همان فرزند عوض می‌شد و با تمام شدن آن، شل (پدر) همچنان در پوشه‌ی قبلی می‌ماند و تغییری دیده نمی‌شد. پس باید در خود پردازه‌ی شل اجرا شود.

بخش ششم: مراحل بوت xv۶

سوال 12: چه فایلی در سکتور اول دیسک قرار می‌گیرد و از چه فایل‌هایی ساخته می‌شود؟

فایل **`bootblock`** در سکتور اول (سکتور صفر) تصویر دیسک `xv6.img` قرار می‌گیرد (با `dd if=bootblock of=xv6.img conv=notrunc`). این فایل از **`bootasm.S`** و **`bootmain.c`** ساخته می‌شود. مراحل در `Makefile` و خروجی `make -n` این است:

```
gcc ... -c bootmain.c   ;   gcc ... -c bootasm.S
ld -m elf_i386 -N -e start -Ttext 0x7C00 -o bootblock.o bootasm.o bootmain.o
objdump -S bootblock.o > bootblock.asm
objcopy -S -O binary -j .text bootblock.o bootblock
./sign.pl bootblock
```

یعنی اول هر دو فایل کامپایل می‌شوند و با آدرس شروع `0x7C00` لینک می‌شوند، بعد قسمت `.text` به صورت خام بیرون کشیده می‌شود و در آخر `sign.pl` فایل را تا ۵۱۰ بایت پر می‌کند و دو بایت امضای `0x55 0xAA` را اضافه می‌کند تا BIOS آن را سکتور بوت معتبر بشناسد.

سوال 13: فایل `bootblock` چه نوع فایلی است و چه فرقی با ELFهای دیگر دارد؟ محتوای `bootblock.o` را با `bootasm.S` مقایسه کنید.

`bootblock` یک **فایل دودویی خام** (**flat binary**) و دقیقاً ۵۱۲ بایت است؛ دستور `file` آن را `DOS/MBR boot sector` می‌شناسد. فایل‌های دیگر مثل `kernel` و `_ls` فایل **ELF** هستند، یعنی هدر ELF، جدول program header، سکشن‌های متعدد، جدول symbol و اطلاعات debug دارند. در bootblock هیچ‌کدام نیست و فقط بایت‌های دستورهای ماشین (به علاوه‌ی امضای `55 AA`) وجود دارد.

علت استفاده از این قالب این است که BIOS هیچ درکی از ELF ندارد. فقط ۵۱۲ بایت اول دیسک را در آدرس `0x7C00` کپی می‌کند و از همان بایت اول اجرا می‌کند، پس بایت اول باید خود کد باشد، نه هدر. ضمناً هدر ELF و اطلاعات اضافی در ۵۱۲ بایت جا نمی‌شد (کد bootblock در این بیلد ۴۵۱ بایت، یعنی `0x1c3`، است).

با `objdump -h bootblock.o` می‌بینیم سکشن `.text` در آدرس `0x7c00` است. خروجی `objdump -S bootblock.o` کد اسمبلی را همراه خط‌های منبع نشان می‌دهد و با `bootasm.S` مطابق است:

```
bootasm.S:                      bootblock.asm (objdump):
  start:                          00007c00 <start>:
    cli                             7c00:  fa            cli
    xorw %ax,%ax                    7c01:  31 c0         xor %eax,%eax
    movw %ax,%ds                    7c03:  8e d8         mov %eax,%ds
    movl %cr0,%eax                  7c22:  0f 20 c0      mov %cr0,%eax
    orl  $CR0_PE,%eax               7c25:  66 83 c8 01   or $0x1,%ax
    ljmp $(SEG_KCODE<<3),$start32   7c2c:  ea 31 7c 08 00 ljmp $0x8,$0x7c31
```

تفاوت‌ها: در `bootblock.asm` برای هر دستور آدرس و بایت‌های ماشین هم آمده و `#define`ها (مثل `CR0_PE` و `SEG_KCODE`) به مقدار عددی تبدیل شده‌اند. نکته‌ی جالب اینکه `objdump` پیش‌فرض کد را ۳۲ بیتی دیس‌اسمبل می‌کند ولی این بخش کد ۱۶ بیتی (real mode) است، برای همین مثلاً `xorw %ax,%ax` به صورت `xor %eax,%eax` و خط `lgdt` در ادامه به شکل چند دستور نامربوط نشان داده می‌شود. بایت‌ها همان‌اند و فقط نمایششان متفاوت است.

سوال 14: چرا هنگام make از objcopy استفاده می‌شود؟

`ld` خروجی را به صورت ELF تولید می‌کند، ولی bootblock باید فقط کد خام باشد. دستور `objcopy -S -O binary -j .text` فقط محتوای سکشن `.text` را بیرون می‌کشد و هدرها، symbolها و اطلاعات debug (با `-S`) را دور می‌ریزد. بدون آن فایل از ۵۱۰ بایت بیشتر می‌شد و `sign.pl` خطا می‌داد و BIOS هم نمی‌توانست مستقیم اجرایش کند. همین کار برای `initcode` و `entryother` هم انجام می‌شود.

سوال 15: چرا نمی‌توان Bootloader را مستقیماً با کد C شروع کرد؟ نقش bootasm.S و bootmain.c چیست؟

پردازنده بعد از روشن شدن در **حالت ۱۶ real mode بیتی** است و هیچ stack تنظیم نشده و ثبات‌های سگمنت مقدار دلخواه دارند. کد C (که برای ۳۲ بیت کامپایل می‌شود) به stack، حالت ۳۲ بیتی و سگمنت‌های درست نیاز دارد. این کارها را فقط می‌شود با دستورهای خاص پردازنده (مثل `lgdt` و تغییر `cr0` و `ljmp`) انجام داد که از C در دسترس نیست. پس اول باید کد اسمبلی اجرا شود.

- **`bootasm.S`**: وقفه‌ها را غیرفعال می‌کند، ثبات‌های `ds`، `es`، `ss` را صفر می‌کند، خط A۲۰ را فعال می‌کند (تا بتوان بیشتر از ۱ مگابایت آدرس داد)، GDT را بارگذاری می‌کند، با ست کردن بیت `PE` در `cr0` وارد protected mode می‌شود، با `ljmp` به کد ۳۲ بیتی می‌رود، ثبات‌های سگمنت را مقدار می‌دهد، stack را روی `0x7c00` می‌گذارد و `bootmain` را صدا می‌زند.
- **`bootmain.c`**: هسته را از دیسک (از سکتور ۱ به بعد) می‌خواند. اول هدر ELF را در آدرس `0x10000` می‌خواند، `magic` را چک می‌کند، سگمنت‌های برنامه (program header) را در آدرس فیزیکی `paddr` کپی می‌کند، قسمت اضافی را صفر می‌کند و در آخر به `entry` هسته می‌پرد.

سوال 16: مد حفاظت‌شده (Protected Mode) چه وظیفه‌ای دارد؟ پردازنده‌ها چه زمانی در این مد قرار می‌گیرند؟

پردازنده‌های x۸۶ برای سازگاری با گذشته با **real mode** شروع به کار می‌کنند که ۱۶ بیتی است، حداکثر ۱ مگابایت حافظه را آدرس‌دهی می‌کند و هیچ حفاظتی ندارد (هر کد به همه‌چیز دسترسی دارد). مد protected mode این امکانات را می‌دهد: آدرس‌دهی ۳۲ بیتی (تا ۴

گیگابایت)، حفاظت حافظه (سگمنت‌ها و صفحه‌بندی)، سطوح دسترسی (۰ ring تا ۳ برای جدا کردن هسته و برنامه‌ی کاربر) و ترجمه‌ی آدرس منطقی به فیزیکی. بدون آن سیستم‌عامل چندوظیفه‌ای امن ممکن نیست.

پردازنده بعد از روشن شدن (reset) در real mode است و **وقتی بوت‌لودر بیت `PE` ثبات `cr۰` را یک کند** و بعد `ljmp` بزند، وارد protected mode می‌شود. در xv۶ این کار در `bootasm.S` و قبل از صدا زدن `bootmain` انجام می‌شود.

سوال 17: چرا هسته در آدرس فیزیکی `0x100000` بارگذاری می‌شود و نه آدرس صفر؟ نقش `0x10000` و `0x100000` در بوت چیست؟

آدرس‌های پایین حافظه در PC برای چیزهای دیگری رزرو هستند: جدول بردار وقفه‌ی BIOS و ناحیه‌ی داده‌ی BIOS در ابتدای حافظه، خود بوت‌لودر در `0x7c00`، و بازه‌ی `0xA0000` تا `0xFFFFF` (حافظه‌ی ویدیو، ROM و BIOS) که روی RAM نیست و حافظه را قبل از ۱ مگابایت تکه‌تکه می‌کند. حافظه‌ی پیوسته و آزاد از `0x100000` (۱ مگابایت، شروع extended memory) به بعد است، برای همین هسته را آنجا می‌گذارند. در ضمن اگر هسته از صفر بارگذاری می‌شد، داده‌های BIOS را خراب می‌کرد.

- **`0x10000`**: فقط یک جای موقت (scratch) است که `bootmain` ۴ کیلوبایت اول فایل هسته را (که هدر ELF در آن است) می‌خواند تا بتواند program headerها را بخواند. هیچ کد هسته‌ای آنجا نمی‌ماند.
- **`0x100000`**: آدرس فیزیکی نهایی هسته (`paddr` در ELF که لینکر `kernel.ld` آن را با `AT(0x100000)` ثبت کرده است. محتوای سگمنت‌های هسته اینجا کپی می‌شود و `entry` هم (حدود `0x10000c`) آنجاست. بعد از فعال شدن صفحه‌بندی، آدرس مجازی `0x80100000` به همین آدرس فیزیکی نگاشت می‌شود.

بخش هفتم: اشکال‌زدایی با GDB

برای اتصال: در یک ترمینال `make qemu-gdb` و در ترمینال دیگر داخل پوشه‌ی xv۶ دستور `gdb` را اجرا می‌کنیم (فایل `.gdbinit` خودکار به QEMU وصل می‌شود و symbolهای `kernel` را می‌خواند).

سوال 20: برای مشاهده‌ی Breakpointها از چه دستوری استفاده می‌شود؟

دستور `info breakpoints` (کوتاه‌شده: `info b` یا `i b`). لیست همه‌ی breakpointها و watchpointها را با شماره، آدرس، محل در کد و تعداد دفعات رسیدن (hit) نشان می‌دهد.

سوال 21: برای حذف یک Breakpoint از چه دستوری و چگونه استفاده می‌شود؟

با `delete N` که `N` شماره‌ی breakpoint در خروجی `info breakpoints` است. مثلاً `delete 2`. اگر بدون شماره نوشته شود (`delete`) همه‌ی breakpointها حذف می‌شوند. همچنین تابع `clear` یا فایل:خط `clear` هم breakpoint همان محل را حذف می‌کند. اگر فقط بخواهیم موقتاً غیرفعال شود از `disable N` استفاده می‌کنیم.

سوال 22: دستور bt چه چیزی را نشان می‌دهد؟

bt (backtrace) زنجیره‌ی فراخوانی توابع تا نقطه‌ی توقف فعلی را چاپ می‌کند: هر خط یک فریم (stack frame) است که شماره‌ی فریم، آدرس، نام تابع، مقدار آرگومان‌ها و فایل و شماره‌ی خط را نشان می‌دهد. فریم #0 تابعی است که الان در آن هستیم و بالاتر، توابع فراخواننده‌اند. مثلاً وقتی در consoleintr متوقف شویم، می‌بینیم از trap و kbdintr فراخوانی شده است.

سوال 23: دو تفاوت x و print را بگویید. چگونه محتوای یک ثبات چاپ می‌شود؟

- x **حافظه** را از یک آدرس می‌خواند و خام نشان می‌دهد، با قالب تعداد قالب اندازه/x مثل x/4xw 0x7c00 (۴ کلمه به صورت hex) یا x/s برای رشته و x/i برای دستور؛ ولی print یک **عبارت یا متغیر** را ارزیابی می‌کند و بر اساس نوعش در برنامه‌ی C چاپ می‌کند (مثلاً کل یک struct را با اسم فیلدها نشان می‌دهد).
- x همیشه آدرس‌ها را کنار مقدار نشان می‌دهد و چند خانه‌ی پشت‌سرهم را می‌خواند؛ print می‌تواند محاسبه یا مقداردهی هم انجام دهد (print a+b، print x=5) و خروجی را در متغیر $1، $2 و ... نگه می‌دارد.

برای چاپ یک ثبات از print $eax یا p/x $esp استفاده می‌شود. همچنین info registers eax هم همین را می‌دهد.

سوال 24: دستورات نمایش ثبات‌ها و متغیرهای محلی چیست؟ کاربرد ESI و EDI چیست؟

برای ثبات‌ها info registers و برای متغیرهای محلی فریم جاری info locals (و برای آرگومان‌ها info args).

ESI (Source Index) و **EDI** (Destination Index) در x۸۶ ثبات‌های عمومی هستند که به‌طور سنتی در دستورهای رشته‌ای و کپی حافظه استفاده می‌شوند. در دستوراتی مثل movsb/movsl و rep movs و rep stos، ثبات ESI آدرس مبدأ و EDI آدرس مقصد را نگه می‌دارد و بعد از هر عملیات خودکار تغییر می‌کنند؛ ECX هم تعداد تکرار است. در xv۶ مثلاً memset در string.c با rep stosb از EDI برای آدرس مقصد استفاده می‌کند و در bootmain.c هم readsect با insl داده را در آدرس EDI می‌ریزد.

info registers خروجی eip، ثبات eax, ecx, edx, ebx, esp, ebp, esi, edi شامل ثبات‌های عمومی است cs, ss, ds, es, fs, gs و ثبات‌های سگمنت eflags، (آدرس دستور بعدی) به‌صورت مقدار هگز و دهدهی. متغیرهایی مثل consoleintr هم متغیرهای محلی تابعی را که در آن توقف کرده‌ایم نشان می‌دهد؛ مثلاً در info locals چاپ می‌شوند (مقدار اولیه‌ی بعضی‌ها قبل از c, doprocdump, lo, hi, oldpos, oldend, i, n, plain, ch mxcsr) همگی صفر (و xmm7 تا xmm0 مقداردهی بی‌معناست. (در اجرای ما، بعد از ثبات‌های عمومی و سگمنت، ثبات‌های هم چاپ شدند 0x1f80 =.

سوال 25: ساختار input در console.c را بررسی کنید.

- **نقش ساختار:** input بافر ورودی کنسول (buf با اندازه‌ی ۱۲۸ بایت) است که کاراکترهای تایپ‌شده را تا زدن Enter نگه می‌دارد. r اندیس خواندن است (جایی که consoleread تا آنجا خوانده)، w اندیس نوشتن یعنی انتهای آخرین خطی که کامل شده و آماده‌ی خواندن است، و e اندیس ویرایش است که انتهای خطی را که کاربر هنوز در حال تایپ آن است نشان می‌دهد. همیشه r ≤ w ≤ e برقرار است.
- **تغییر مقادیر:** در ابتدا هر سه صفرند. با تایپ هر کاراکتر e یکی زیاد می‌شود ولی r و w ثابت می‌مانند (مثلاً بعد از abc: r=0, w=0, e=3). با Enter، کاراکتر \n هم در بافر می‌آید (e=4)، سپس w = e می‌شود و پردازه‌ی منتظر (شل) بیدار می‌شود. وقتی شل با read خطوط را می‌خواند، r جلو می‌رود تا به w برسد (r=w=e=4). با Backspace مقدار e یکی کم می‌شود.

- **متغیرهای جدید ما:** `pos` (اندیس مکان‌نما در بافر، همیشه بین `w` و `e`)، `selstate` (حالت انتخاب: ۰ بدون انتخاب، ۱ در حال انتخاب، ۲ انتخاب‌شده)، `selbeg` و `selend` (نقطه‌ی شروع و پایان انتخاب) و همچنین آرایه‌ی `clip` و `cliplen` برای متن کپی‌شده.
- **بررسی با GDB**: یک breakpoint روی `consoleintr` می‌گذاریم و بعد از تایپ چند کاراکتر مقادیر را بررسی می‌کنیم:

```
(gdb) b consoleintr
(gdb) c            # در QEMU یک کاراکتر تایپ می‌کنیم
(gdb) p input.pos
(gdb) p input.e
(gdb) p input.w
(gdb) p input.selstate
(gdb) x/s input.buf
```

مقادیر، `Enter` بعد از تایپ دو کاراکتر و بدون زدن `consoleintr`، هنگام توقف روی GDB: نتیجه‌ی اجرای واقعی در `input.w = 0` و `input.e = 2` (یعنی دو کاراکتر در بافر ویرایش وجود دارد که هنوز برای شل آماده نشده‌اند `w`) بود؛ و `pos=3`، `e=3`، `w=0` تایپ شده باشد انتظار داریم `abc` ثابت می‌ماند. (مثال دیگر) طبق منطق کد: (اگر `Enter` زدن `c` و `b` همان ۳ بماند، یعنی مکان‌نما بین `e` برابر ۲ شود ولی `pos` باشد و پس از فشردن ← مقدار `selstate=0` است. محتوای بافر تغییر نکرده است.

اشکال‌زدایی در سطح اسمبلی

سوال 26: خروجی دستورات `layout src` و `layout asm` در TUI چیست؟

`layout src` یک پنجره در بالا باز می‌کند که **کد منبع C** فایل فعلی را نشان می‌دهد و خط در حال اجرا و breakpointها را علامت می‌زند. `layout asm` به جای آن **کد اسمبلی** (دیس‌اسمبل) تابع فعلی را نشان می‌دهد و دستورالعمل فعلی علامت دارد. پایین هر دو پنجره‌ی خط فرمان GDB است. (با `Ctrl+X` و بعد `A` می‌توان TUI را روشن یا خاموش کرد.)

سوال 27: برای جابه‌جایی بین توابع زنجیره‌ی فراخوانی از چه دستوراتی استفاده می‌شود؟

با `up` یک فریم به سمت فراخواننده می‌رویم، با `down` به سمت فراخوانده‌شده برمی‌گردیم و با `frame N` (یا `f N`) مستقیم به فریم شماره‌ی N می‌رویم. در هر فریم می‌توان با `info locals` و `print` متغیرهای همان تابع را دید. دستور `bt` هم شماره‌ی فریم‌ها را نشان می‌دهد.